CounselConnect

Technology Stack, Algorithms & Justifications

Technology Stack & Algorithms

AI-Assisted Counseling Management Platform

Major Project · Team ID: P7_056

Department of Computer Engineering

Faculty of Engineering & Technology

Farhan Sargath · Nakul Dabhi · Kavya Vaghela

Internal Guide: Prof. Priyanka Mangi

Generated from the CounselConnect source code

CounselConnect — Technology Stack, Algorithms & Design Justifications

Companion to `PROJECT-DOCUMENTATION.md` Every library, every algorithm, why it was chosen, and what was rejected.

1. How to use this document

Every technology on your slides is a licence for an examiner to ask "*why did you choose that?*" — and "*why not X instead?*" This document gives you a defensible answer for each.

The strongest viva answers have three parts:

> **1. What it does · 2. Why we chose it · 3. What we rejected and why**

A student who says "*we used bcrypt because it's secure*" scores less than one who says "*we used bcrypt rather than SHA-256 because SHA-256 is fast, and speed is exactly wrong for password hashing — it makes brute-forcing cheap.*"

---

2. Backend Dependencies

2.1 Express `^4.19.2` — Web framework

What it does. Provides HTTP routing, middleware chaining and request/response handling on top of Node's `http` module.

Why we chose it. Its middleware model maps exactly onto our security requirements — each request passes through an ordered pipeline (`helmet → cors → json → authenticate → requireRole → validate → handler`), and each stage can reject before the next runs. Authorisation is therefore structural rather than something a developer must remember to call.

Alternatives rejected:

Alternative	Why not
Fastify	2–3× faster in benchmarks, but our bottleneck is I/O and database access, not routing. Express has far more documentation and community answers, which matters on a student timeline
NestJS	Excellent structure (decorators, DI, modules), but it imposes a heavy TypeScript-first architecture. For a 3-person team learning full-stack, the ceremony would have cost more than it returned
Koa	Cleaner async middleware, but a much smaller ecosystem — several packages we needed (multer, express-validator) are Express-specific
Raw Node `http`	We would have rewritten routing, body parsing and middleware ourselves. No educational gain proportional to the cost

> **Viva:** "*Express is unopinionated, which meant we chose our own layered structure — routes, controllers, services — rather than inheriting a framework's. Our business logic never touches Express request objects, so it could be reused behind a different framework or tested independently.*"

2.2 mongodb `^7.5.0` — Official MongoDB driver

What it does. Native Node.js driver: connection pooling, BSON serialisation, CRUD, bulk operations.

Why we chose the raw driver over Mongoose. This is our most-questioned decision, and it was deliberate.

Our entire backend reads data **synchronously** — 412 call sites across the backend:

```
const appointments = readStore('appointments.json');   // returns immediately
const mine = appointments.filter(a => a.userId === userId);
```

Mongoose is asynchronous throughout. Adopting it would have required converting every one of those 412 call sites, and every function containing them, to `async/await` — propagating through controllers to routes. That is effectively a full backend rewrite, and it would have invalidated every feature we had already tested.

Instead we built a **storage abstraction** (`utils/fileStore.utils.js` + `utils/mongoStore.utils.js`) that keeps the synchronous API while making MongoDB the database of record:

- collections are loaded into memory at boot
- reads are served from that in-process copy (synchronous, as the services expect)

- writes update the copy immediately, then write through to MongoDB on an **ordered queue** so two rapid saves cannot land out of sequence

Alternatives rejected:

Alternative	Why not
Mongoose ODM	Schemas and validation are genuinely valuable, but the async conversion cost was disproportionate. We enforce validation at the API layer with express-validator instead
Prisma	Excellent type safety, but its MongoDB support is less mature than its SQL support, and it requires a schema-first workflow that fights document flexibility
PostgreSQL + Sequelize	Considered seriously. Rejected because our documents genuinely vary in shape — a counselor record has `qualifications`, `availability` and `languages[]` that a client record does not, and mood entries carry variable-length `tags[]`. In SQL this becomes sparse columns or join tables

> **Viva — expect this:** "*Why not Mongoose?*" "*Mongoose is asynchronous and our service layer is synchronous across 412 call sites. Converting all of them would have been a full rewrite of tested code. We used the native driver behind a storage abstraction: MongoDB is the source of truth, loaded at startup, and every write goes to it on an ordered queue. The trade-off is that it assumes a single backend process, which we document. With more time, migrating service-by-service to async Mongoose would be the next step.*"

Honest limitation: the read-through cache means multiple server instances would not see each other's writes. Correct for one process; documented in `BACKLOG.md` as DB1.

2.3 jsonwebtoken `^9.0.2` — Authentication tokens

What it does. Signs and verifies JSON Web Tokens (HS256).

Why we chose it. Our API is stateless and serves three separate panels. A JWT carries the user id **and role** in its signed payload, so `requireRole('doctor')` needs no database lookup — the role is cryptographically attested by the signature.

Alternatives rejected:

Alternative	Why not
express-session + connect-mongo	Server-side sessions require a session store and a lookup on every request. They also complicate horizontal scaling (sticky sessions or shared store)
Passport.js	A strategy framework aimed at many providers (Google, Facebook, LDAP). We have one strategy — email/password — so it would add abstraction without benefit
OAuth 2.0 / Auth0	Appropriate for third-party sign-in, but we need our own credential store, and an external dependency during a live demo is a risk

Our configuration: HS256, 7-day expiry, payload `{ id, role }`, plus an in-memory blacklist so logout invalidates a token before natural expiry.

> **Viva — the honest weakness:** "*We store the token in localStorage, which is vulnerable to XSS. An httpOnly cookie would be safer but needs CSRF protection. For production we would move to httpOnly cookies with CSRF tokens.*" **Naming the weakness earns more credit than hiding it.**

2.4 bcryptjs `^2.4.3` — Password hashing

What it does. Implements the bcrypt adaptive hash — a deliberately slow, salted, one-way function.

Why we chose it. Password hashing must be **slow**. bcrypt's cost factor means each hash takes measurable time, so brute-forcing a stolen database is expensive. Every password gets a unique salt automatically, so identical passwords produce different hashes and rainbow tables are useless.

Our configuration: 12 salt rounds = 2^{12} = 4,096 iterations, roughly 250ms per hash on typical hardware. Fast enough for login, slow enough to make offline attack impractical.

Alternatives rejected:

Alternative	Why not
SHA-256 / MD5	**NO Catastrophically wrong.** Designed to be *fast*. A modern GPU computes billions of SHA-256 hashes per second. MD5 is additionally broken
Argon2	**Technically superior** — won the 2015 Password Hashing Competition, and is memory-hard, so it resists GPU/ASIC attack better. Rejected only because `argon2` requires native compilation, which complicates setup across three developers' machines and any deployment target. bcrypt remains industry-accepted
PBKDF2	Acceptable and FIPS-approved, but not memory-hard, so it is weaker than bcrypt against GPU attack
`bcrypt` (native)	Same algorithm, faster C++ implementation, but needs `node-gyp` and build tools. `bcryptjs` is pure JavaScript — zero install friction, at ~30% of the speed. Irrelevant for our request volume

> **Viva:** "*Why bcrypt and not SHA-256?*" "*SHA-256 is a fast general-purpose hash — exactly wrong for passwords, because speed helps the attacker. bcrypt is deliberately slow with a tunable cost factor and per-password salting. We used 12 rounds. Argon2 is theoretically stronger because it is memory-hard, but it needs native compilation, which we avoided for portability.*"

2.5 socket.io `^4.8.1` — Real-time communication

What it does. Bidirectional event-based communication with automatic reconnection, room-based broadcast, and fallback to HTTP long-polling.

Why we chose it. Three features need to *push* data to the client:

1. **Chat** — messages must arrive without polling
2. **Presence** — "online now" must be genuine
3. **WebRTC signaling** — offer/answer/ICE must relay in real time

Alternatives rejected:

Alternative	Why not
Raw WebSocket (`ws`)	Lighter, but we would implement reconnection, heartbeats, rooms and event multiplexing ourselves — that is most of Socket.IO
HTTP polling	We *do* keep polling as a fallback, but as the primary transport it means either high latency or wasteful requests. Unusable for call signaling, where a delayed SDP answer breaks the handshake
Server-Sent Events	One-directional (server→client). Our chat and signaling need client→server too
Firebase Realtime DB / Pusher	Managed and reliable, but adds a third-party dependency, ongoing cost, and puts private counseling messages on someone else's infrastructure — a poor fit for mental-health data

How we use it: each authenticated socket joins a room named `user:<id>` or `doctor:<id>`, so a message is delivered to an *account* rather than a socket — meaning it reaches every tab that account has open.

2.6 PDFKit `^0.19.1` — PDF generation

What it does. Programmatic PDF construction, streamed directly to the HTTP response with no temporary files.

Why we chose it. We generate **seven** documents: journal summary, appointment summary, appointment details, counseling note, practice report, mood report and patient list. All are data-driven and branded, and none are ever written to disk — the PDF streams straight to the browser.

Alternatives rejected:

Alternative	Why not
Puppeteer (HTML→PDF)	Renders HTML/CSS beautifully, but launches a headless Chromium per request — ~300MB and 1–2 seconds each. Disproportionate for a table-based report
jsPDF (client-side)	Generation would happen in the browser, meaning the client needs all the data. For clinical reports the data must stay server-side and be access-controlled
pdfmake	Declarative and pleasant, but PDFKit gives finer control over the page-footer pass we need (`bufferPages` + `flushPages`) for "Page 1 / 3" numbering

Two implementation details worth mentioning in viva: - `bufferPages: true` is required to write footers on *every* page — otherwise `switchToPage()` only ever sees the last one. - `doc.page.margins.bottom = 0` before drawing the footer, or PDFKit spawns a blank page for content that overflows the margin.

2.7 Multer `^1.4.5-lts.1` — File uploads

What it does. Express middleware parsing `multipart/form-data`.

Why we chose it. It is the de-facto Express standard and integrates as ordinary middleware, so file routes stay in the same pipeline as everything else.

Our security configuration:

```
limits:     { fileSize: 25 * 1024 * 1024, files: 1 }
fileFilter: /\.(pdf|docx?|png|jpe?g|webp|txt|csv|xlsx?)$/i
storage:    DOC_DIR   // OUTSIDE the statically-served uploads folder
```

> **Important design point:** clinical documents are stored **outside** the public `uploads/` directory and served only through an authenticated route that checks the counselor-patient relationship. Avatars *are* public, so they live under the static folder. **This split is a strong viva point.**

Alternatives rejected: `formidable` / `busboy` (lower-level, more manual); `express-fileupload` (simpler API but less control over storage and filtering).

2.8 express-validator `^7.1.0` — Input validation

What it does. Declarative validation and sanitisation as route middleware.

Why we chose it. Validation lives *with the route definition*, so the contract is visible where the endpoint is declared:

```
router.post('/',
  [ body('value').isInt({min:1, max:5}).withMessage('Mood value must be 1–5'),
    body('label').notEmpty().withMessage('Mood label is required') ],
  validate, ctrl.logMood);
```

Alternatives rejected: Joi and **Zod** are both excellent — Zod especially, since it infers TypeScript types. Rejected because they validate a *value*, so each route needs boilerplate to extract and pass `req.body`. express-validator plugs directly into the middleware chain. *(If we were TypeScript end-to-end on the backend, Zod would be the better choice.)*

2.9 Helmet `^7.1.0` — Security headers

Sets `Content-Security-Policy`, `X-Frame-Options` (clickjacking), `X-Content-Type-Options` (MIME sniffing), `Strict-Transport-Security` and removes `X-Powered-By`. One line, several classes of attack mitigated — there is no reason not to use it.

2.10 cors `^2.8.5`

The frontend (`:5173`) and API (`:5000`) are different origins, so the browser blocks requests by default. We allow-list the specific frontend origin rather than using `*`, which would let any site call our API with credentials.

2.11 dotenv `^16.4.5`

Keeps `JWT_SECRET`, `MONGODB_URI` and ports out of source control. **A hardcoded JWT secret in a committed repository is a real vulnerability** — anyone with the source could forge tokens for any user.

2.12 uuid `^9.0.1`

Generates RFC 4122 v4 identifiers. Chosen over auto-increment because IDs are created in the application layer before insertion, and sequential IDs leak information (how many users you have, and they are enumerable). *Considered:* `nanoid` (smaller, URL-friendly) — uuid was already familiar and the difference is immaterial at our scale.

2.13 nodemon `^3.1.4` *(dev)*

Restarts the server on file change. Development only.

---

3. Frontend Dependencies

3.1 React `18` — UI library

Why. Three panels share components (cards, tables, modals, charts). React's component model and one-way data flow make that reuse safe, and its declarative rendering suits dashboards that update from polling and WebSocket events.

Alternatives rejected:

Alternative	Why not
Vue 3	Genuinely comparable and arguably gentler to learn. React chosen for the larger ecosystem and better employability
Angular	Batteries-included (DI, routing, forms, HTTP), but a steep learning curve and heavier bundle. Its opinionation would help a large team; ours is three people
Svelte	Compiles away the framework, giving the smallest bundle and excellent performance. Rejected for a smaller ecosystem and less hiring-relevant experience
Vanilla JS	Managing three role-based dashboards with live updates by hand would be error-prone and slow

3.2 TypeScript — Type safety

Catches type errors at author time, and gives editor autocomplete across ~80 component files.

> **Honest disclosure:** the project has **no `tsconfig.json` and no type-checking build step**. Vite strips types without checking them, so our TS annotations act as documentation and editor assistance, not enforcement. **Say this if asked — do not claim compile-time safety we do not have.** Adding `tsc --noEmit` to the build is listed in `BACKLOG.md` as X1.

3.3 Vite `6.3.5` — Build tool

Why. Dev server starts in under a second and applies Hot Module Replacement instantly, because it serves native ES modules rather than bundling on every change.

Alternatives rejected: Create React App (deprecated, slow, Webpack-based); **Webpack** (powerful but complex configuration and much slower rebuilds); **Parcel** (zero-config but a smaller plugin ecosystem).

3.4 Tailwind CSS `4.1.12` — Styling

Why. Utility classes keep styling next to markup, so a component is self-contained, and the design stays consistent because spacing and colour come from one scale.

Alternatives rejected: Bootstrap (recognisably "Bootstrap-looking" without heavy overrides); **styled-components** (runtime CSS-in-JS cost); **plain CSS/SCSS** (naming discipline and dead-CSS management across 80 components); **MUI** as the primary system (opinionated Material aesthetic; we wanted a calmer palette appropriate to mental health).

3.5 React Router `7.13.0`

Client-side routing with nested layouts — the dashboard shell renders once and only the outlet changes. Route guards redirect by role.

3.6 socket.io-client `4.8.1`

Must match the server major version. Wrapped in a **singleton** so the whole app shares one connection:

> **A bug worth mentioning:** tearing down a socket that is merely *connecting* orphans every listener bound to it, silently killing an in-flight call. Our `getSocket()` only rebuilds the connection when the auth token changes.

3.7 Recharts `2.15.2` — Charts

Declarative React components (`<LineChart>`, `<Bar>`) rather than imperative canvas calls, with `<ResponsiveContainer>` handling resize.

Alternatives rejected: Chart.js (imperative, needs refs and manual lifecycle in React); **D3.js** (maximum power, but we need standard chart types and D3 would fight React for DOM control); **ApexCharts** (good, but Recharts is React-native and lighter for our needs).

3.8 Motion (Framer Motion) `12.23.24`

Declarative animation — page transitions, staggered list entry, modal enter/exit via `<AnimatePresence>` (which CSS cannot do for unmounting elements).

3.9 lucide-react `0.487.0`

Tree-shakeable SVG icons — only imported icons ship. Chosen over Font Awesome (icon-font requests, less crisp) and Material Icons (Material aesthetic).

3.10 Radix UI (~30 packages) + shadcn/ui

Why. Radix provides **unstyled, accessible** primitives — dialogs, dropdowns, tabs, tooltips — with keyboard navigation, focus trapping and ARIA handled correctly. shadcn/ui is not a dependency but a set of components copied into `admin/components/ui/` (48 files), so they are ours to modify.

Why not build our own? Accessible modals and menus are genuinely hard — focus trap, escape handling, scroll lock, `aria-*` wiring, click-outside. Radix solves that correctly.

3.11 Supporting libraries

Library	Purpose
`date-fns` `3.6.0`	Date formatting — modular and tree-shakeable, unlike Moment.js (large, mutable, deprecated)
`react-hook-form` `7.55.0`	Uncontrolled form state — avoids a re-render per keystroke
`clsx` + `tailwind-merge`	Conditional class names, with correct Tailwind conflict resolution
`sonner`	Toast notifications
`@mui/material`	Used in isolated places; **overlaps with Radix/shadcn — a legitimate criticism, worth acknowledging as accumulated inconsistency**
`canvas-confetti`	Success celebration
`next-themes`	Dark mode

> **Expect this question:** "*Why do you have both MUI and Radix?*" **Honest answer:** "*The admin panel was built on shadcn/Radix and some earlier components used MUI. Consolidating on one is technical debt we have identified but not yet paid down.*" That is a better answer than inventing a justification.

---

4. Algorithms

4.1 WARNING Counselor matching — READ THIS BEFORE YOUR VIVA

File: `backend/services/ai.service.js` → `computeMatch()`

There is a problem here you must fix or disclose

```
const computeMatch = (counselor, answers) => {
  let score = Math.floor(Math.random() * 20) + 75;   // <span class="tag warn">WARNING</span> RANDOM 75–94
  // …then +5 if the counselor's approach matches a preferred style
  // …then +8 if a concern keyword matches their specialty
  return score;
};
```

The match percentage shown to users is seeded with `Math.random()`. Keyword matching only adjusts it by +5 or +8. This means:

- The same user answering the same questionnaire twice gets **different percentages**
- A counselor with **no** relevant specialty can outscore one with a perfect match, purely by random draw
- `buildReason()` likewise picks one of four templates **at random**, so the stated reason may not describe why that counselor actually ranked highly

This affects `POST /api/ai/match` (the AI Matching questionnaire page).

`GET /api/ai/recommended` — used by the dashboard — is better: it derives real inputs from the user's `reason`, `sessionType`, `goals` and their two lowest-scoring mood tags. **But it then calls the same `matchCounselors()`, so its displayed percentage inherits the randomness too.** The *selection basis* is real; the *number* is not.

Why this matters more than any other issue

Your project is titled "AI-Powered". An examiner asking "*how is the match percentage calculated?*" and being shown `Math.random()` would undermine the credibility of everything else in the presentation.

The fix (small — roughly 20 lines)

Replace the random base with a deterministic weighted score:

```
const computeMatch = (counselor, answers) => {
  let earned = 0, possible = 0;
  const spec = String(counselor.specialty || '').toLowerCase();
  const appr = String(counselor.approach  || '').toLowerCase();

  // 1. Concern ↔ specialty  (weight 40)
  possible += 40;
  if (answers[0] && CONCERN_MAP[answers[0]]?.some(k => spec.includes(k))) earned += 40;

  // 2. Preferred style ↔ approach  (weight 25)
  possible += 25;
  const styles = String(answers[1] || '').split(', ').filter(Boolean);
  if (styles.some(s => appr.includes(s.toLowerCase().split(' ')[0]))) earned += 25;

  // 3. Goal keyword overlap  (weight 20)
  possible += 20;
  const goalHits = answers.slice(2).filter(g => spec.includes(String(g).toLowerCase().split(' ')[0]));
  earned += Math.min(20, goalHits.length * 10);

  // 4. Standing — rating normalised to 0–15  (weight 15)
  possible += 15;
  earned += ((Number(counselor.rating) || 0) / 5) * 15;

  return Math.round((earned / possible) * 100);   // deterministic, explainable
};
```

Then you can say, truthfully: "*Match score is a weighted sum across four factors — presenting concern against specialty at 40%, preferred style against approach at 25%, goal overlap at 20%, and counselor rating at 15% — normalised to a percentage. It is deterministic and fully explainable.*"

Ask me and I will implement and test this.

The honest description of your AI, as it stands today

> "*Our AI layer is rule-based and statistical rather than machine-learned. Counselor recommendation builds a profile from the user's declared concern, session-type preference, goals and the context tags attached to their lowest mood entries, then ranks counselors by keyword overlap with their specialisation. Mood analysis performs trend detection over time-series entries. The journey summary is generated from computed statistics, not a language model. We chose a deterministic approach because it is explainable and auditable, which matters in a mental-health context, and because we do not ethically have training data. LLM integration is documented future work.*"

Rehearse that answer. It is honest, technically precise, and turns the limitation into a reasoned design decision.

4.2 TOTP — Two-factor authentication (RFC 6238) KEY

File: `backend/utils/totp.utils.js` — **implemented from the specification, with no library.**

This is the strongest algorithmic work in the project. Lead with it.

The algorithm

```
1. Enrolment    generate 160-bit secret (crypto.randomBytes(20))
                encode base32 → shown as a QR code (otpauth:// URI)

2. Code generation, every 30 seconds:
   counter = floor(unixTime / 30)              time step
   buf     = counter as 8-byte big-endian
   hmac    = HMAC-SHA1(base32Decode(secret), buf)     → 20 bytes

3. Dynamic truncation (RFC 4226 §5.3):
   offset  = hmac[19] & 0x0F                   low 4 bits pick a start byte
   code    = ((hmac[offset]   & 0x7F) << 24)   mask the sign bit
           | ((hmac[offset+1] & 0xFF) << 16)
           | ((hmac[offset+2] & 0xFF) <<  8)
           |  (hmac[offset+3] & 0xFF)
   otp     = code mod 10^6, zero-padded to 6 digits

4. Verification: accept steps −1, 0, +1  (±30s clock drift tolerance)
                 compare with crypto.timingSafeEqual
```

Three details that demonstrate real understanding

1. **Dynamic truncation** — the last nibble of the HMAC selects *where* to read the 4 bytes. This is why the whole 160-bit hash contributes entropy rather than a fixed slice.
2. **The `0x7F` mask** clears the high bit so the value is unambiguously positive across platforms with signed 32-bit integers.
3. **`timingSafeEqual`** compares in constant time. A normal `===` returns faster on an early mismatch, which leaks information — an attacker can time responses to discover the code digit by digit.

Verified against the RFC 6238 published test vectors — e.g. secret `12345678901234567890` at T=59s produces `287082`, and at T=1111111109s produces `081804`. Our implementation reproduces both.

Why implement rather than use `speakeasy` or `otplib`? Educational value, zero dependency, and it is fully auditable. For production, a maintained library would be the safer default.

> **Viva:** "*Why HMAC-SHA1 and not SHA-256?*" "*RFC 6238 permits SHA-256 and SHA-512, but SHA-1 is what authenticator apps universally implement, so interoperability with Google Authenticator requires it. The known SHA-1 collision attacks affect collision resistance, not HMAC security, so HMAC-SHA1 remains sound here.*"

4.3 WebRTC call establishment KEY

Files: `src/app/lib/callClient.ts`, `backend/realtime/signaling.js`

Your most technically impressive feature.

The handshake

```
1. Caller  getUserMedia() → local audio/video tracks
2. Caller  emit call:invite → server checks canConnect() → relays call:incoming
3. Callee  accepts → emit call:accept → server relays call:accepted
4. Caller  createOffer() → setLocalDescription() → emit webrtc:offer
5. Callee  setRemoteDescription(offer) → createAnswer()
           → setLocalDescription() → emit webrtc:answer
6. Both    ICE candidates gathered and exchanged via webrtc:ice
7. ICE agents test candidate pairs → connectivity established

   ═══ MEDIA FLOWS DIRECTLY BROWSER ↔ BROWSER ═══
   Audio and video never touch the server
```

Engineering problems solved (each is a viva talking point)

Problem	Solution
Offer lost in transit	Re-send the offer up to 4 times at 3s intervals, stopping once the remote description is applied
ICE arrives before the remote description	Queue candidates in `pendingIce[]`, flush after `setRemoteDescription()` — applying early throws
Duplicate offer	Cache the answer and re-send it rather than renegotiating a stable connection
Both state flags unreliable	`connectionState` sometimes never leaves "connecting" on the answering side. We treat `iceConnectionState`, `connectionState` *or* actual media arrival as success
A brief drop froze the UI permanently	Separate "has ever connected" from "connected right now", so recovery re-emits `connected` — previously the guard prevented it and the remote video stayed black forever
Autoplay blocked silently	Call `play()` explicitly and surface a "Tap to start video" fallback rather than showing a black rectangle with no error
Nobody answers	45-second ring timeout with a clear message

STUN, and the honest limitation

We use Google's public STUN servers for NAT traversal. STUN discovers your public IP so peers can address each other directly.

> **Expect:** "*What if the peer-to-peer connection fails?*" "*Symmetric NATs and restrictive corporate firewalls can block direct connections. The production answer is a TURN relay server, which forwards media when direct connection is impossible. We have not deployed one — it requires bandwidth and hosting — so we detect the failure and show a clear message rather than leaving the user waiting. TURN is documented future work.*"

Why not Zoom SDK / Agora / Twilio / Stream? They would have been faster to integrate, but they cost money per minute, put private counseling sessions on third-party infrastructure, and would have taught us nothing about how real-time media actually works. Building the signaling ourselves was the point.

4.4 Bookable slot generation

File: `backend/services/counselors.service.js` → `getCounselorSlots()`

```
FOR each of the next 14 days:
    IF date falls inside the counselor's vacation range → skip
    rule ← weekly schedule for that weekday
    IF day disabled or has no windows → skip
    FOR each window (start, end):
        FOR minute m FROM start TO end STEP 60:
            IF slot time ≤ now                    → skip (already past)
            IF slot overlaps the break window     → skip
            booked ← any non-cancelled appointment at that exact time
            emit { time, iso, booked }
```

A timezone bug worth describing. The day bucket key was originally built with `toISOString().slice(0,10)`. `toISOString()` converts to UTC, so in IST (UTC+5:30) any local time before 05:30 belongs to the *previous* UTC day. The client looked up the wrong bucket and was offered times that had already passed. Fixed by constructing the key from local date components:

```
const localKey = (d) => {
  const off = d.getTimezoneOffset() * 60000;
  return new Date(d.getTime() - off).toISOString().slice(0, 10);
};
```

> This is an excellent answer to "*what was the hardest bug you fixed?*" — it shows understanding of timezones, which trips up most developers.

4.5 Mood trend analysis

File: `backend/services/ai.service.js`

Before/after growth by life area:

```
FOR each context tag with ≥ 4 entries:
    split the user's entries for that tag into first half / second half
    before ← mean(first half)  × 20    → percentage
    after  ← mean(second half) × 20
    delta  ← after − before
```

Design decision: it returns an **empty array** when there is insufficient data, rather than inventing dimensions. An earlier version fabricated five life areas by offsetting one weekly average by fixed amounts (−14, +10, …), producing plausible-looking numbers that meant nothing. Replacing that with honest emptiness is a deliberate choice worth stating.

8-week history: exact 7-day windows ending today, labelled with each week's real start date — not calendar weeks, which would make the current partial week look artificially low.

4.6 Password reset by OTP

File: `backend/services/passwordReset.service.js`

```
1. POST /forgot-password { email }
   → ALWAYS respond "if that account exists, a code has been sent"
     (identical response and timing whether or not the account exists —
      prevents account enumeration)
   → if it exists: generate a 6-digit code, store hashed with a 10-minute expiry

2. POST /verify-reset-code { email, code }
   → max 5 attempts, then invalidate
   → 60-second resend cooldown

3. POST /reset-password { email, code, newPassword }
   → re-verify, hash with bcrypt, invalidate the code
```

Account enumeration is the attack this defends against: if the endpoint said "no such user", an attacker could discover which email addresses are registered. **In a mental-health platform, merely revealing that someone has an account is a privacy breach.** That framing is worth saying out loud.

4.7 Availability enforcement

File: `backend/services/availability.service.js` — the single authority for when a counselor is bookable, used by both slot generation and the booking guard.

```
checkBookable(counselorId, when):
    IF onVacation(settings, when)      → reject, name the date range
    rule ← schedule[weekday(when)]
    IF day disabled                    → reject
    IF slot not inside a window        → reject
    IF slot overlaps the break         → reject
    ELSE ok

At booking time:
    IF vacation           → always reject (409)
    ELSE IF autoReject on → reject immediately with the reason
    ELSE                  → allow, but flag `outsideHours` so the counselor
                            sees why the request is unusual
```

A bug worth telling: the counselor's saved schedule had no effect on what clients could book. The Availability page wrote to `availability.json`, but the slot generator read `doctor.availability` — a seed **string** (`'Mon–Fri'`), not an object. Spreading a string into an object produces indexed characters, so the defaults silently won and every counselor offered the same 9–5. One shared service now owns the logic.

4.8 MongoDB storage engine

File: `backend/utils/mongoStore.utils.js`

```
BOOT:      connect → load all 21 collections into memory
READ:      serve from the in-process copy (synchronous)
WRITE:     1. update the copy immediately  (readable at once)
           2. enqueue a MongoDB write on an ordered promise chain
              bulkWrite(replaceOne upsert) + deleteMany({_id: {$nin: keep}})
SHUTDOWN:  flush the queue before exiting
```

Why upsert-plus-delete rather than drop-and-insert? A drop followed by an insert leaves the collection **empty** if the process dies between the two. Upserting and then deleting what is no longer present converges on the same result without ever passing through an empty state.

Content-hash keys. Rows without their own `id` — the login history — are keyed by an FNV-1a hash of their content rather than array index. `logins` keeps only the last 200 entries, so index keys would shift on every login and rewrite all 200 documents.

4.9 Relationship-based access control

File: `backend/services/relationship.service.js`

Role checks are not sufficient here. A counselor is authorised to use counselor features, but **not** to read *any* patient's data — only their own.

```
canConnect(personA, personB):
    resolve both to (userId, counselorId)
    return TRUE if they share any appointment
           OR an existing message thread
```

Enforced on chat, calls, journals, documents and patient records.

> **Viva:** "*How do you stop one counselor reading another's patient notes?*" "*Role-based access control authorises the counselor role, but authorisation is also relationship-scoped:* `canConnect()` *verifies the two parties share an appointment or conversation before any patient data is returned. Role alone would let any counselor read any patient.*"

---

5. Architectural Decisions

5.1 Layered backend (routes → controllers → services)

```
routes/       URL + validation rules
controllers/  HTTP request/response only
services/     business logic — NEVER touches req/res
utils/        storage, JWT, hashing, PDF
```

Services can be tested without an HTTP server, and business rules are not entangled with Express. Our integration tests call services directly.

5.2 One React app, three panels

Why not three separate applications? One codebase, one build, one deployment, shared components and API client. Role separation is enforced by route guards **and** server-side role checks — never by shipping separate bundles, since a client-side-only guard is not security.

5.3 Consistent response envelope

Every endpoint returns `{ success, message, data }`, so the client handles success and failure identically everywhere.

5.4 REST + WebSocket together

REST for request/response (fetch appointments, submit a mood). WebSocket for push (a message arriving, a call ringing). Using REST for chat would mean polling; using WebSocket for everything would lose HTTP caching, status codes and straightforward debugging.

---

6. Technologies Deliberately NOT Used

Be ready for "why didn't you use…?"

Technology	Why not
Redux / Zustand	Our shared client state is small — the signed-in user and theme. React Context covers it. Redux would add boilerplate for no benefit
GraphQL	Solves over/under-fetching for many clients with varied needs. We have one client and stable views; REST is simpler to debug and cache
Docker	Would help deployment consistency, but adds a learning curve and our target was a local demo
Redis	Would be the right cache/session store at scale. Our in-process cache is sufficient for one instance
Kubernetes / microservices	Enormous operational overhead for a project with one backend. A monolith is the correct architecture at this size
OpenAI / Gemini API	Cost per request, needs internet during a demo, and sending mental-health conversations to a third party raises real privacy concerns. Our rule-based approach is explainable and auditable
Cloudinary / S3	Managed media storage with CDN. Local storage was sufficient and avoids an external account and cost. Listed as future work
Stream / Agora / Twilio Video	Paid per-minute video SDKs. Building WebRTC signaling ourselves cost more time but taught far more and has no per-minute cost. WARNING **Your slide 11 incorrectly lists Stream — correct it**
Python / TensorFlow / scikit-learn	Would be required for genuine ML. We have no labelled training data — and building a clinical recommendation model on invented data would be irresponsible
Jest / Mocha	WARNING **We have no committed automated test suite.** Testing was manual plus scripted integration checks. Be honest about this
Nginx	Reverse proxy / TLS termination — a deployment concern we have not reached

7. Quick-Reference Justification Table

One line each — memorise these.

Technology	Chosen because	Instead of
Express	Middleware pipeline makes auth structural	Fastify (speed we don't need), NestJS (too heavy for 3 people)
MongoDB	Documents vary in shape between roles	PostgreSQL (sparse columns / join tables)
Native driver	Our 412 service calls are synchronous	Mongoose (would force a full async rewrite)
JWT	Stateless; role travels in the signed payload	Sessions (server store + lookup per request)
bcrypt (12 rounds)	Deliberately slow, per-password salt	SHA-256 (too fast), Argon2 (needs native build)
Socket.IO	Reconnection, rooms and fallback built in	Raw WebSocket (rebuild all of it), polling (too slow for signaling)
WebRTC	Media stays peer-to-peer — private and free	Stream/Agora (per-minute cost, 3rd-party privacy)
PDFKit	Streams to the response, no temp files	Puppeteer (300MB Chromium per request)
Multer	Standard Express middleware; type + size limits	formidable (lower level)
express-validator	Validation sits with the route definition	Joi/Zod (need per-route boilerplate)
React	Component reuse across three panels	Angular (steep), Svelte (smaller ecosystem)
Vite	Sub-second start, instant HMR	CRA (deprecated), Webpack (slow rebuilds)
Tailwind	Consistent scale, styles beside markup	Bootstrap (generic look), CSS-in-JS (runtime cost)
Recharts	Declarative React charting	Chart.js (imperative), D3 (fights React for the DOM)
Radix/shadcn	Accessibility done correctly	Hand-rolled (focus traps and ARIA are hard)
TOTP hand-written	Educational, zero dependency, auditable	speakeasy/otplib (better for production)

8. Before Your Viva — Priority Actions

#	Action	Why
1	**Fix `computeMatch()`** (§4.1) or be ready to disclose the randomness	Single biggest credibility risk
2	**Correct "Stream" → WebRTC + Socket.IO** on slide 11	You do not use Stream, and the truth is more impressive
3	**Rehearse the AI answer** (§4.1)	Your most likely hostile question
4	**Memorise:** 12 salt rounds · 21 collections · 180 endpoints · HS256 · 7-day expiry · ±1 TOTP window	Specific numbers signal genuine familiarity
5	**Prepare the TURN answer** (§4.3)	"What if P2P fails?" is a standard follow-up
6	**Be honest about testing**	Claiming a test suite you cannot show is worse than admitting manual testing
7	**Have the timezone bug story ready** (§4.4)	Best answer to "hardest bug you solved"

Every version number, file path and code excerpt in this document was read directly from the CounselConnect source at the time of writing.